

AI에게 규칙과 기억을 주고, 결과를 확인하는 방법

`coding-agent-setting.md` 에서 도구를 설치했다면 이 문서가 그 다음이다. 여기서 다루는 것은 `gotchas.md`, `lessons-learned.md`, 하네스(harness) 세 가지다.

0. 이 문서가 푸는 문제

AI에게 코드를 시키면 거의 항상 이렇게 답한다.

"완료했습니다. 코드가 정상적으로 동작합니다."

이 문장은 실행 결과가 아니라 글자다. AI는 코드를 실행하지 않고도 이 문장을 쓸 수 있다. 실제로 자주 그렇게 한다. 그래서 확인하는 쪽은 사람이어야 한다.

강의계획서의 감점 대상에 이 문제가 그대로 들어 있다.

- 실행하지 않은 결과를 제출
- 오류 메시지를 숨김
- AI가 만든 코드를 이해하지 못한 채 제출

세 가지 모두 "AI 말을 그대로 믿었다"에서 나온다. 이 문서는 믿는 대신 확인하는 방법을 다룬다.

1. 세 가지가 각각 무엇인가

이름	한 줄 정의	언제 적는가	이 저장소의 실물
<code>gotchas.md</code>	다시 밟으면 안 되는 함정 목록	같은 실수가 두 번째 나왔을 때	<code>.ai/gotchas.md</code>
<code>lessons-learned.md</code>	문제 → 해결 → 규칙을 남기는 기록	한 번 푼 문제를 다시 풀지 않으려고	<code>.ai/lessons-learned.md</code>
하네스	말 대신 파일과 로그로 확인하는 장치	코드를 고칠 때마다	<code>scripts/run_and_capture.py</code> , <code>scripts/course_gates.py</code> , <code>harness.md</code>

셋의 관계는 이렇다. gotchas는 "하지 마라"의 목록이고, lessons-learned는 "왜 그렇게 정했나"의 기록이고, 하네스는 "정말 그렇게 했는지" 확인하는 프로그램이다.

앞의 둘은 AI가 읽고 참고할 뿐이라서 지킬 수도 있고 안 지킬 수도 있다. 하네스만 지켰는지를 판정한다.

2. 하네스라는 말

harness는 원래 장비를 몸에 고정하는 끈을 뜻한다. 소프트웨어에서는 뜻이 좁아져서, 코드를 실제로 돌려 보고 결과가 기준에 맞는지 자동으로 확인하는 프로그램을 test harness라고 부른다. AI에게 일을 시킬 때도 같은 뜻으로 쓴다.

규칙 파일과 하네스는 역할이 다르다.

	규칙 파일 (AGENTS.md)	하네스 (scripts/*.py)
하는 일	AI에게 무엇을 하라고 적어 둔다	했는지 검사하고 결과를 숫자로 낸다
안 지키면	아무 일도 일어나지 않는다	종료코드 1이 나오고 실패로 기록된다
사람이 할 일	한 번 써 두기	명령 한 줄 실행하기

그래서 작업 환경은 세 층으로 쌓는다.

1층	규칙	AGENTS.md	— AI에게 이 프로젝트의 규칙을 알려 준다
2층	기억	gotchas.md	— 같은 실수를 반복하지 않게 한다
		lessons-learned.md	— 한 번 내린 결정을 다시 뒤집지 않게 한다
3층	증거	하네스	— 1·2층을 지켰는지 파일로 확인한다

1층과 2층만 있으면 AI가 규칙을 어겨도 아무도 모른다. 3층이 있어야 어긴 것이 드러난다.

3. 이 수업 저장소에서 직접 해 보기

교수자 저장소를 클론했다면 하네스가 이미 들어 있다. 클론한 폴더에서 터미널을 열고 실행한다.

```
python scripts/run_and_capture.py 5 --verify
```

5장 실습 코드가 지금도 유효한지 검사하는 명령이다. 이렇게 나온다.

```
[verify] chapter5 - 3개 파일
[ok] 유효: 5-0-concept-figures.py
[ok] 유효: 5-1-one-neuron.py
[ok] 유효: 5-2-xor.py
검증 완료: 문제 0건
```

3-1. 무엇을 보고 유효하다고 판정했나

practice/chapter5/results/ 폴더를 열면 로그 파일 옆에 .evidence.json (증거 파일)이 있다. 5-1-one-neuron.evidence.json 의 실제 내용에서 주요 항목만 뽑은 것이다.

```
{
  "file": "5-1-one-neuron.py",
  "started_at": "2026-08-24T20:41:52.310452+09:00",
  "duration_sec": 1.96,
  "exit_code": 0,
  "success": true,
  "python_version": "3.12.7",
  "platform": "Windows-11-10.0.26200-SP0",
  "source_sha256": "3f8967e69d8c39f1ab066a36f3f3fa70f81140fdf15cd0418e45395ae64ad330",
  "output_log": "practice\\chapter5\\results\\5-1-one-neuron.log"
}
```

항목	뜻	왜 남기나
exit_code, success	프로그램이 오류 없이 끝났는가	실패한 실행을 성공이라고 쓰지 못하게
started_at, duration_sec	언제 몇 초 걸렸나	결과가 언제 것인지 알려고
python_version, platform	어느 기계에서 돌렸나	환경이 달라 결과가 달라질 때 원인을 찾으려고
source_sha256	그때 코드의 지문	코드를 고치면 이 값이 달라진다

source_sha256 이 핵심이다. 코드를 한 글자만 고쳐도 이 값이 바뀌고, --verify 가 [stale] 낡은 결과 를 낸다. 코드를 고치고 결과는 옛날 것을 그대로 쓰는 일을 막는 장치다.

직접 확인해 보려면 5장 코드 아무 파일 끝에 주석 한 줄(# 시험)을 넣고 --verify 를 다시 실행한다. [stale] 이 나온다. 주석을 지우면 다시 [ok] 가 된다.

3-2. 교재 본문의 숫자는 어디서 왔나

docs/ch5.md 에서 실행 결과가 나오는 자리 아래를 보면 이런 줄이 붙어 있다.

```
_출처: `practice/chapter5/results/5-1-one-neuron.log`_
```

교재의 모든 수치에 이 표기가 붙는다. 그 숫자가 어느 로그 파일의 몇 번째 실행에서 나왔는지 되짚을 수 있다는 뜻이다. 되짚을 수 없는 숫자는 교재에 넣지 않는다.

저장소 전체를 한 번에 검사하는 명령도 있다.

```
python scripts/course_gates.py
```

4. 내 과제 폴더에 최소 하네스 만들기

과제 저장소에도 같은 구조를 만든다. 파일 세 개와 폴더 두 개면 된다.

교수자 저장소를 클론한 폴더(`basic_DL`) 안에 만들지 않는다. 내 문서 폴더에 따로 폴더를 하나 만들고 거기에 둔다.

클론한 폴더 안에 내 파일을 넣으면, 교수자 자료를 새로 받을 때(`git stash` → `git pull` → `git stash pop`) 내 작업물이 같이 떨어 들어간다.

```
내 문서/
├─ basic_DL/           교수자 저장소를 클론한 폴더 (여기는 건드리지 않는다)
└─ my-project/         내가 만드는 과제 폴더
    ├─ AGENTS.md       규칙 (coding-agent-setting.md 5절 참고)
    ├─ gotchas.md       함정 목록
    ├─ lessons-learned.md  해결 기록
    ├─ code/            내가 쓴 코드
    └─ results/         실행 로그와 결과 파일
```

이 폴더는 한 학기에 하나만 만들어 계속 쓴다. 과제마다 새로 만들지 않는다. `gotchas.md` 와 `lessons-learned.md` 는 한 학기 동안 쌓여야 쓸모가 생긴다. 2주차에 GitHub 저장소를 만들 때 이 폴더를 그대로 올리면 된다.

4-1. gotchas.md 틀

```
# Gotchas - 다시 밟지 말 것

## 환경
- 가상환경을 켜지 않고 pip install 하면 다른 곳에 깔린다. 먼저 켜다.

## 데이터
- CSV를 읽을 때 encoding="utf-8"을 안 주면 한글이 깨진다.

## AI 사용
- AI가 준 코드에 내 파일 경로가 하드코딩되어 있을 때가 있다. 실행 전에 경로를 먼저 본다.
```

두 번 겪은 것만 적는다. 한 번도 안 겪은 일을 미리 적으면 목록이 길어져서 아무도 안 읽는다.

4-2. lessons-learned.md 틀

이 저장소의 `.ai/lessons-learned.md` 가 쓰는 형식을 그대로 가져온 것이다.

```
## 2026-09-16: 3장 실습 - 그래프에 한글이 네모로 나옴

### 상황
matplotlib으로 그린 그래프의 한글 라벨이 전부 네모로 나왔다.

### 해결책
폰트를 맑은 고딕으로 지정했다.

### 규칙
- 그래프를 만들면 저장한 PNG를 반드시 열어서 눈으로 본다.
```

날짜, 무슨 일이 있었나, 어떻게 풀었나, 다음부터 어떻게 하나. 네 가지면 된다.

4-3. 실행 로그 남기기

화면에 뜬 결과를 캡처하는 대신 파일로 남긴다. 오류 메시지까지 같이 저장된다.

Windows (PowerShell)

```
mkdir results -Force
python code\my_code.py *> results\run.log
```

macOS / Linux

```
mkdir -p results
python3 code/my_code.py > results/run.log 2>&1
```

제대로 됐는지는 이렇게 확인한다. `results` 폴더에 `run.log` 가 생겼는가. 그 파일을 열었을 때 화면에 나왔던 내용이 그대로 들어 있는가. 오류가 났다면 오류 메시지도 들어 있는가. 셋 다 맞으면 성공이다.

4-4. 제출 전 점검

점검	확인 방법
코드가 실제로 돌아다	<code>results/run.log</code> 가 있고 내용이 들어 있다
로그가 지금 코드의 것이다	코드를 고친 뒤 다시 실행했다
오류를 숨기지 않았다	실패한 로그도 지우지 않고 남겼다
AI 사용을 적었다	강의계획서의 AI 활용 기록 다섯 줄을 채웠다

5. AI에게 뭐라고 말하나

작업을 시키기 전

작업 시작 전에 `AGENTS.md`와 `gotchas.md`를 읽고, 이 프로젝트에서 하지 말아야 할 것 세 가지를 먼저 말해 줘.

작업이 끝났다고 할 때

"완료"라고만 하지 말고, 실제로 실행한 명령과 `results/run.log`의 마지막 10줄을 그대로 보여 줘.
로그가 없으면 완료라고 하지 말고 실행부터 해 줘.

같은 실수가 또 나왔을 때

방금 그 실수는 지난번에도 나왔어. `gotchas.md`에 한 줄로 추가해 줘. 무엇을 하지 말아야 하는지만 짧게.

6. 과제·프로젝트와 어떻게 연결되나

언제	요구하는 것	하네스로 어떻게 대응하나
매주 실습	실습을 실행하면 만들어지는 결과 파일	<code>results/</code> 폴더의 파일을 그대로 제출
12주차 (AI 코딩 에이전트)	도구 코드 + 실행 로그 + 보안 점검표	4-3의 방법으로 남긴 로그
기말 프로젝트	"결과가 실제로 실행되었음을 보여준다" (필수 요건 4번)	로그 파일 + 실행 화면
기말 프로젝트	AI 도구 사용 기록 (필수 요건 3번)	<code>lessons-learned.md</code> 가 그대로 기록이 된다

7. 자주 하는 실수

실수	왜 문제인가	대신 할 것
화면만 캡처하고 로그를 안 남긴다	캡처는 오려 붙일 수 있고 오류가 잘린다	파일로 남긴다
코드를 고친 뒤 옛 로그를 제출한다	로그와 코드가 다른 것을 말한다	고쳤으면 다시 실행한다
실패한 실행을 지운다	실패도 결과다. 숨기면 감점 대상이다	남기고 원인을 적는다
gotchas에 안 겪은 일을 적는다	목록이 길어져 아무도 안 읽는다	두 번 겪은 것만 적는다
AI에게 "완료?"라고만 묻는다	AI는 실행하지 않고도 완료라고 답한다	명령과 로그를 보여 달라고 한다